

E02 — Sort an Array (Merge Sort / Quick Sort)

Experiment: 2 | LeetCode: #912 | CO: CO1, CO5 | BT Level: BT4

Problem Statement

Given an array of integers `nums`, sort the array in ascending order and return it.

You must solve the problem **without using any built-in sort functions** in $O(n \log n)$ time and with the smallest space complexity possible.

Example 1:

Input: `nums = [5, 2, 3, 1]`

Output: `[1, 2, 3, 5]`

Example 2:

Input: `nums = [5, 1, 1, 2, 0, 0]`

Output: `[0, 0, 1, 1, 2, 5]`

Solution 1: Merge Sort

Concept

Divide and Conquer:

1. **Divide:** Split array into two halves.
2. **Conquer:** Recursively sort each half.
3. **Combine:** Merge the two sorted halves.

Implementation

```
def sortArray_merge(nums):  
    """  
    Merge Sort.  
    Time:  $O(n \log n)$  all cases | Space:  $O(n)$  for merge buffer  
    """  
    if len(nums) <= 1:  
        return nums  
  
    def merge_sort(arr):  
        if len(arr) <= 1:  
            return arr  
  
        mid = len(arr) // 2
```

```

        left = merge_sort(arr[:mid])
        right = merge_sort(arr[mid:])
        return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

return merge_sort(nums)

def sortArray_merge_inplace(nums):
    """In-place merge sort using index ranges (O(log n) stack space)."""
    def merge_sort(arr, lo, hi):
        if hi - lo <= 0:
            return
        mid = (lo + hi) // 2
        merge_sort(arr, lo, mid)
        merge_sort(arr, mid + 1, hi)
        merge_in_place(arr, lo, mid, hi)

    def merge_in_place(arr, lo, mid, hi):
        left = arr[lo:mid+1]
        right = arr[mid+1:hi+1]
        i = j = 0
        k = lo
        while i < len(left) and j < len(right):
            if left[i] <= right[j]:
                arr[k] = left[i]; i += 1
            else:
                arr[k] = right[j]; j += 1
            k += 1
        while i < len(left):
            arr[k] = left[i]; i += 1; k += 1
        while j < len(right):
            arr[k] = right[j]; j += 1; k += 1

    merge_sort(nums, 0, len(nums) - 1)
    return nums

```

Solution 2: Quick Sort (Randomized)

Concept

Divide and Conquer:

1. **Partition:** Pick a pivot; rearrange so $\text{left} < \text{pivot} \leq \text{right}$.
2. **Recurse:** Sort left and right parts.

Randomized pivot avoids worst-case $O(n^2)$ on sorted inputs.

Implementation

```
import random

def sortArray_quick(nums):
    """
    Randomized Quick Sort.
    Time:  $O(n \log n)$  average,  $O(n^2)$  worst | Space:  $O(\log n)$  avg (stack)
    """
    def quick_sort(arr, lo, hi):
        if lo < hi:
            pivot_idx = partition(arr, lo, hi)
            quick_sort(arr, lo, pivot_idx - 1)
            quick_sort(arr, pivot_idx + 1, hi)

    def partition(arr, lo, hi):
        # Randomize pivot to avoid worst case
        rand_idx = random.randint(lo, hi)
        arr[rand_idx], arr[hi] = arr[hi], arr[rand_idx]

        pivot = arr[hi]
        i = lo - 1

        for j in range(lo, hi):
            if arr[j] <= pivot:
                i += 1
                arr[i], arr[j] = arr[j], arr[i]

        arr[i + 1], arr[hi] = arr[hi], arr[i + 1]
        return i + 1

    quick_sort(nums, 0, len(nums) - 1)
    return nums
```

3-Way Partition (Dutch National Flag — handles duplicates well)

```
def sortArray_3way(nums):
    """
    3-way quick sort — efficient when many duplicate elements.
    """
    def quick3(arr, lo, hi):
        if lo >= hi:
            return
        # Partition into < pivot | == pivot | > pivot
```

```

    lt, gt = lo, hi
    pivot = arr[lo]
    i = lo + 1

    while i <= gt:
        if arr[i] < pivot:
            arr[lt], arr[i] = arr[i], arr[lt]
            lt += 1; i += 1
        elif arr[i] > pivot:
            arr[gt], arr[i] = arr[i], arr[gt]
            gt -= 1
        else:
            i += 1

    quick3(arr, lo, lt - 1)
    quick3(arr, gt + 1, hi)

quick3(nums, 0, len(nums) - 1)
return nums

```

Detailed Trace

Merge Sort on [5, 2, 3, 1]

```

Split: [5, 2, 3, 1]
├─ Split: [5, 2]
│   ├─ [5] (base case)
│   └─ [2] (base case)
│   Merge [5] and [2] → [2, 5]
└─ Split: [3, 1]
    ├─ [3] (base case)
    └─ [1] (base case)
    Merge [3] and [1] → [1, 3]

```

```

Merge [2, 5] and [1, 3]:
  Compare 2 vs 1: take 1 → [1]
  Compare 2 vs 3: take 2 → [1, 2]
  Compare 5 vs 3: take 3 → [1, 2, 3]
  Remaining: 5 → [1, 2, 3, 5] ✓

```

Quick Sort Partition on [3, 6, 8, 10, 1, 2, 1], pivot=3 (index 0)

```

pivot = 3
i = -1

j=0: arr[0]=3 ≤ 3 → i=0, swap arr[0]↔arr[0]: [3, 6, 8, 10, 1, 2, 1]
j=1: arr[1]=6 > 3 → no swap
j=2: arr[2]=8 > 3 → no swap
j=3: arr[3]=10 > 3 → no swap
j=4: arr[4]=1 ≤ 3 → i=1, swap arr[1]↔arr[4]: [3, 1, 8, 10, 6, 2, 1]
j=5: arr[5]=2 ≤ 3 → i=2, swap arr[2]↔arr[5]: [3, 1, 2, 10, 6, 8, 1]

```

```
j=6: arr[6]=1 ≤ 3 → i=3, swap arr[3]↔arr[6]: [3, 1, 2, 1, 6, 8, 10]
```

```
Final swap arr[i+1]↔arr[hi]: arr[4]↔arr[6] → ...
```

```
(pivot at index 4; left [3,1,2,1] all ≤ 3, right [8,10] all > 3)
```

Complexity Comparison

Algorithm	Best	Average	Worst	Space
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick Sort (random)	$O(n \log n)$	$O(n \log n)$	$O(n^2)$ unlikely	$O(\log n)$
Quick Sort (3-way)	$O(n)$ (all same)	$O(n \log n)$	$O(n^2)$	$O(\log n)$

Test Suite

```
def test_sort():
    solutions = [sortArray_merge_inplace, sortArray_quick, sortArray_3w

    test_cases = [
        ([5, 2, 3, 1], [1, 2, 3, 5]),
        ([5, 1, 1, 2, 0, 0], [0, 0, 1, 1, 2, 5]),
        ([1], [1]),
        ([], []),
        ([2, 2, 2], [2, 2, 2]),
        ([-5, 3, -1, 0, 2], [-5, -1, 0, 2, 3]),
    ]

    for fn in solutions:
        print(f"\n{fn.__name__}:")
        for nums, expected in test_cases:
            result = fn(nums[:]) # Copy to avoid mutation
            status = "PASS" if result == expected else "FAIL"
            print(f"    {status}: {nums[:5]}... → {result[:5]}...")

test_sort()
```

Key Takeaways

- **Merge Sort:** Guaranteed $O(n \log n)$, stable, but uses $O(n)$ extra space.
 - **Quick Sort:** $O(n \log n)$ average with randomization; $O(1)$ extra space (in-place); cache-friendly.
 - **3-Way Quick Sort:** Best when input has many duplicate values (degenerates to $O(n)$ for all-equal).
 - For this LeetCode problem, either merge sort or randomized quick sort is acceptable.
-

References

- LeetCode #912 — Sort an Array
- Cormen et al., *Introduction to Algorithms*, Ch. 7 (Quicksort), Ch. 2.3 (Merge Sort)